Thamill Indian B U
Cohort-15 (AI Engineer)

Day 16 Post-Session Assignment & Engineering Reflection

AI Engineering Program | Day 16: AI Observability & Debugging with LangSmith

Git Repo: AI-Assignments/Day16 at main · ThamillIndianBlend360/AI-Assignments

1. Challenge Output & Repository Guide

This repository contains full end-to-end telemetry, custom tracing wrappers, PII redaction hooks, fault-injection flags, and evaluation scripts for the Day 16 Observability challenge.

Directory & Artifact Map (day16/)

- **src/tracer.py:** Custom LangSmith wrapper initializing client, dataset hooks, metadata injection, and automated masking pre-execution hooks.
- **src/assistant.py:** Core AI assistant workflow with wrapped `@traceable` nodes for retriever, LLM judge, prompt formatter, and output guardrails.
- **src/faults.py:** Fault injection manager supporting configurable toggles (`--fault-latency`, `--fault-hallucination`, `--fault-corrupt-context`).
- **src/masking.py:** Regex and NER hybrid masking engine for scrubing sensitive personal details (PII) before span emission.
- **Day16/judge.py:** Traced evaluation harness running Day 15 benchmark suite against the assistant and logging trace IDs.
- **Day16/evaluation_results_before.jsonl:** Baseline trace evaluation run output containing 30 test cases prior to architectural fix.
- **Day16/evaluation_results_after.jsonl:** Re-evaluation run output confirming 100% pass rate (Level 3 quality) post-fix.

Execution Instructions

To execute the assistant and running evaluation harnesses with tracing enabled, run the following commands from the project root:

```
# 1. Environment Setup & LangSmith Enablement
$env:LANGCHAIN_TRACING_V2="true"
$env:LANGCHAIN_API_KEY="<your_private_langsmith_key>"
$env:LANGCHAIN_PROJECT="day16-observability-cohort15"

# 2. Standard Execution with Full Tracing
python Day16/judge.py --app-version v1.2.0

# 3. Executing Fault Scenarios
python Day16/judge.py --fault-latency  # Simulates 15s vector store retrieval delay
python Day16/judge.py --fault-hallucination  # Injects corrupted prompt context
```

2. Decisions & Architectural Trade-offs

When implementing tracing across an enterprise agentic pipeline, capturing everything creates excessive noise and cost, while capturing too little makes debugging impossible. Our instrumentation strategy balances high actionable visibility with compliance and efficiency.

Tracing & Metadata Decisions

- **Traced Granularity:** Captured spans covering four discrete operational steps: `Retriever.search()`, `PromptFormatter.build()`, `LLM.generate()`, and `Guardrail.validate()`. Every span is explicitly typed (`retriever`, `llm`, `parser`, `tool`).
- **Trace Metadata:** Every trace carries `case_id` (e.g., D15-001), `model_name` (llama3), `app_version` (v1.2.0), `environment` (dev/eval), and `user_hash`. This enables instant filtering of failing cases in LangSmith dashboard.
- **PII Redaction:** Client-side regex scrubbers strip email addresses, phone numbers, and full names before sending payload spans to LangSmith endpoint.

Trade-off & Alternative Analysis

Architectural Choice	Advantages	Disadvantages / Trade-offs
1. Tracing Granularity (Full Spans vs. Top-Level Only)	Full visibility into internal retriever query formatting, prompt construction, and raw LLM responses.	Higher payload size, slightly higher API logging latency (~15ms), larger storage consumption.
2. PII Handling (Client-Side Masking vs. Cloud Scrubbing)	Ensures zero raw PII ever leaves local perimeter or hits external observability cloud endpoints.	Potential false-positive redaction on technical strings; minor CPU overhead prior to span emission.

3. Root Cause Diagnoses

Through LangSmith trace analysis across synthetic fault runs and baseline evaluations, two distinct systemic defects were isolated and diagnosed.

Diagnosis 1: Quality Failure (Hallucinated Policy Bounds)

- **Case ID & Symptom:** D15-019 (Multi-entity compliance check).
- **Failing Step:** LLM Output Generation step (`llm_generate` span).
- **Trace Evidence:** LangSmith trace `run-id: b8f3a91c-2026-4a11-8812` showed retriever fetched 4 relevant document chunks, but the system prompt builder truncated chunk #3 due to a hardcoded string slice limit `context[:500]`.
- **Root Cause:** The LLM received incomplete context regarding edge-case governance exceptions, forcing the model to hallucinate standard enterprise fallbacks.

Diagnosis 2: Latency & Timeout Failure (Retriever Bottleneck)

- **Case ID & Symptom:** D15-001 (Cold start evaluation query).
- **Failing Step:** Vector Search Step (`retriever_query` span).
- **Trace Evidence:** Trace showed a 40.343s total execution duration, with 38.2s spent inside vector store connection setup and embedding model warm-up.
- **Root Cause:** Un-cached DB connection instantiation inside the request loop, causing synchronous cold-start overhead on initial request.

4. Fix Implementation & Re-Evaluation Suite

To resolve the quality bottleneck (Diagnosis 1), the prompt formatting logic was updated from a rigid character slice to token-budget aware dynamic context packaging. Connection pooling was also added to mitigate vector store cold start latency.

Day 15 Evaluation Suite Comparative Benchmark (Before vs. After)

Case ID	Before Fix Verdict (Time)	After Fix Verdict (Time)	Delta / Outcome
D15-001	Level 1 Fail (40.34s - Latency/Cold)	Level 3 Pass (1.82s)	Resolved Cold Start (-95.5%)
D15-002	Level 3 Pass (17.45s)	Level 3 Pass (14.21s)	Stable Performance
D15-003	Level 3 Pass (18.70s)	Level 3 Pass (15.10s)	Slight Speedup
D15-010	Level 2 Partial (17.93s)	Level 3 Pass (13.88s)	Improved Structure
D15-019	Level 1 Fail (21.10s - Context Cut)	Level 3 Pass (14.65s)	Quality Fix Confirmed
D15-020	Level 3 Pass (16.46s)	Level 3 Pass (14.12s)	Stable Performance
D15-025	Level 3 Pass (13.21s)	Level 3 Pass (12.05s)	Stable Performance
D15-028	Level 3 Pass (14.37s)	Level 3 Pass (13.11s)	Stable Performance
D15-030	Level 3 Pass (18.68s)	Level 3 Pass (14.90s)	Stable Performance
Summary (D15-001..30)	**28/30 Level 3 (Avg 18.2s)**	**30/30 Level 3 (Avg 14.8s)**	**100% Pass Rate / +18.6% Faster**

Note: Zero test cases degraded post-fix. The context packaging update resolved the edge-case truncated retrieval quality drops without introducing regressions across the remaining 29 test cases.

5. Failures, Telemetry & FDE Governance

Trace Blindspots & Observability Limits

While LangSmith provided full execution path visibility, two specific factors remained opaque:

- **Local Hardware Contention:** Quantized model weights running locally on Ollama/Llama3 suffered from GPU memory throttling that appeared in traces merely as slow execution duration without explicit internal stack traces.
- **Vector Index Drift:** When retrieval returned low relevance scores, LangSmith captured the returned documents, but could not determine if missing context was due to bad embedding alignment or actual document absence in the vector database.

Telemetry Decision & Rate Limits

- **Collected vs. Dropped:** Collect execution duration, token usage, case ID, retriever score distributions, and sanitised outputs. Drop raw user inputs containing unverified PII.
- **Access & Retention:** Traces accessible only to AI Engineering and Platform Ops teams. Retained for 30 days in development; 7 days in production.
- **Volume & Budget:** A full evaluation run across 30 cases generates exactly 150 span objects (~350KB payload), representing under 0.35% of standard monthly free-tier observability limits.

FDE Lens (Forward Deployed Engineering)

From an FDE standpoint, observability must strictly serve operational reliability without compromising enterprise privacy or inflating cloud cost overhead. Collecting metadata-heavy, content-light traces provides 90% of debugging value at <5% of compliance risk.

6. Engineering Reflection & Next Steps

If given one additional hour on this implementation, the highest-priority improvement would be:

- **Real-time Context Relevancy Evaluator:** Automate an asynchronous evaluator directly inside the LangSmith callback loop that scores context relevancy against chunk density using a lightweight judge model. This would detect context truncation in real time during live execution before the response reaches the user.